



GAN : Generative Adversarial Networks

1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

1. Introduction

논문이 다루는 분야

- 딥러닝에서의 성공
 - 데이터에 대한 확률 분포를 나타내는 풍부하고 계층적인 모델 발견하는 것
 - 판별 모델에서의 성공

해당 task에서 기존 연구 한계점

- 딥 생성 모델의 어려움
 - 최대 가능도 추정 및 관련 전략에서 발생하는 많은 추적 불가능한 확률 계산의 근사
 - 생성 맥락에서 piecewise linear unit의 장점 활용 어려움

논문의 contributions

- **adversarial nets framework**
 - 생성 모델과 판별 모델의 대립
 - 생성모델이 데이터 샘플을 생성하도록 학습, 판별모델은 샘플이 모델 분포에서 왔는지 데이터분포에서 왔는지를 결정하도록 학습
 - 다양한 종류의 모델과 최적화 알고리즘에 대한 특정 훈련 알고리즘 도출 가능
 - 본 논문에서는 생성 모델과 판별 모델 모두 다층 퍼셉트론 — 적대적 신경망 (adversarial nets)

- backpropagation 및 dropout 알고리즘만을 사용해 모델 훈련 가능, 생성모델로부터 순방향 전파만을 사용해 샘플링 가능 (근사 추론이나 마르코프 연쇄 X)



Swin Transformer: Hierarchical Vision Transformer using Shifted Windows

1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

2. Related Work

3. 제안 방법론

Main Idea

Contribution

4. 실험 및 결과

Dataset

Baseline

결과

5. 결론 (배운점)

1. Introduction

논문이 다루는 분야

- 컴퓨터 비전: CNN 아키텍처의 발전, 백본 네트워크로 사용됨
- 자연어 처리: 트랜스포머 아키텍처 — 이후 컴퓨터 비전에 적용
- 본 논문은 트랜스포머의 적용 범위를 확장해 컴퓨터비전을 위한 범용 백본으로 확대하고자 함

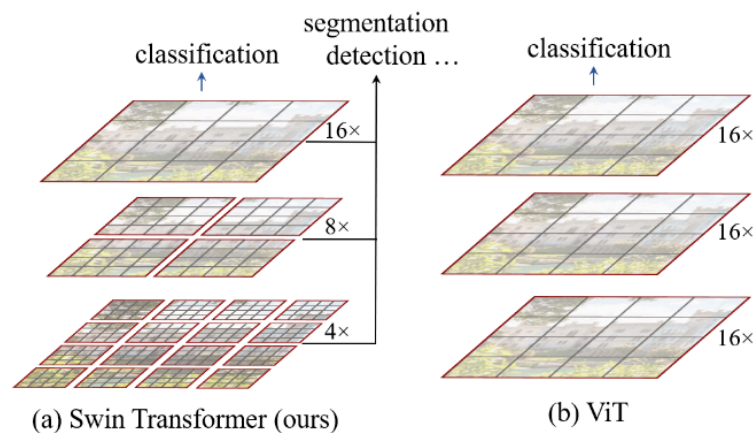
해당 task에서 기존 연구 한계점

- 언어 도메인에서 시각 도메인으로 — 두 모달리티간의 차이로 인한 어려움
 - 스케일 차이: 단어 토큰과 달리 시각적 요소들은 스케일이 다를 수 있음
 - 이미지 픽셀의 높은 해상도 - 계산 복잡도가 이차이므로 고해상도 이미지에서 실현 어려움

⇒ 계층적 특징 맵 구축, 이미지 크기에 대해 선형 계산 복잡도 갖는 범용 트랜스포머 백본 **Swin Transformer** 제안

논문의 contributions

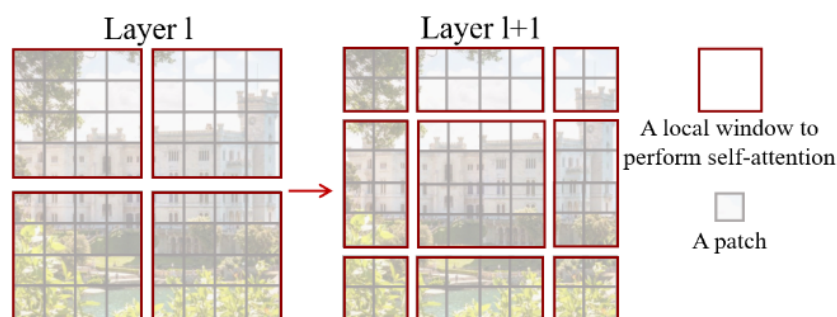
Swin Transformer



- 작은 크기의 패치에서 시작, 더 깊은 트랜스포머 레이어에서 이웃 패치를 병합해 계층적 표현 구축
 - 이런 계층적 특징 맵을 통해 feature pyramid networks(FPN)이나 U-Net 같은 dense prediction을 편리하게 사용 가능
- 이미지를 분할하는 non-overlapping 레이어 내에서 지역적으로 self-attention 계산해 선형 계산 복잡성을 달성
 - 각 창의 패치 수는 고정되어 있어 복잡성이 이미지 크기에 대해 선형이 됨

→ 단일 해상도의 특징 맵 생성, 이차적 복잡성 갖는 이전 Transformer 기반 아키텍처에 대한 장점

- Swin Transformer 구조



- 연속적인 셀프 어텐션 레이어 간의 윈도우 분할 이동(shifted window)
 - 이동된 윈도우는 이전 레이어의 윈도우를 연결, 윈도우 간의 연결 제공으로 성능 향상
 - 윈도우 내의 모든 쿼리 패치는 동일한 키 세트 공유 → 하드웨어에서 메모리 접근 용이, 지연 시간 효율적
 - 이전의 슬라이딩 윈도우 기반 셀프 어텐션 방식은 서로 다른 쿼리 픽셀에 대해 서로 다른 키 세트 사용 → 낮은 지연 시간
 - shifted window는 sliding window 방식보다 비슷한 성능으로 더 낮은 지연 시간 보임

- Swin Transformer는 이미지 분류, 객체 탐지, semantic segmentation과 같은 인식 작업에서 높은 성능 달성
 - 유사한 지연 시간으로 ViT/DeiT, ResNe(X)t 모델을 능가
 - 다양한 데이터셋 및 태스크에서 높은 정확도와 성능 보임
- Swin Transformer와 같이 컴퓨터 비전과 자연어 처리 전반에 걸친 통합된 아키텍처가 두 분야 모두에 이익이 될 수 있다 믿음

2. Related Work

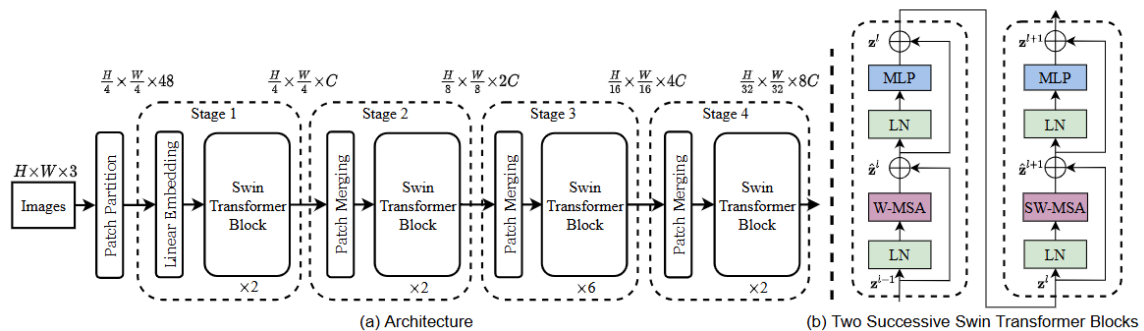
- **CNN**
 - 컴퓨터 비전 전반에 걸친 표준 네트워크 모델
 - 다양한 깊고 효과적인 컨볼루션 신경망 아키텍처의 제안 및 개선 연구
- **Self-attention 기반 백본 아키텍처**
 - ResNet의 일부를 대체해 셀프 어텐션 레이어 사용
 - 각 픽셀의 로컬 윈도우 내에서 셀프 어텐션 계산 — 비용 ↑, 지연시간 ↑
 - 논문에서는 슬라이딩 윈도우 대신 shifted window 제안 → 효율적 구현 가능
- **CNN을 보완하기 위한 Self-attention/Transformers**
 - 표준 CNN 아키텍처에 셀프 어텐션 레이어나 트랜스포머를 추가
 - 셀프 어텐션을 통한 distant dependencies, heterogeneous interactions 인코딩
 - 트랜스포머의 인코더-디코더 설계 적용
- **Transformer 기반 비전 백본**

- ViT 및 후속 연구(DeiT 등)
- Swin Transformer는 분류에만 국한되지 않고 일반적인 성능에 초점 맞춘 연구

3. 제안 방법론

Main Idea

- Swin Transformer architecture



1. 패치 분할 모듈 사용해 입력 RGB 이미지를 겹치지 않는 패치로 분할
 - 각 패치는 토큰으로 취급, feature는 raw pixel RGB 값의 연결로 설정
 - 4×4 패치 크기 $\rightarrow$ feature dimension은 $4 \times 4 \times 3 = 48$
 - 이 raw value feature에 선형 임베딩 레이어를 적용해 임의의 차원 C 로 투영
2. Swin Transformer 블록을 패치 토큰에 적용
 - 트랜스포머 블록은 토큰 수를 유지($H \ 4 \times W \ 4$)
 - 선형 임베딩과 함께 묶어 Stage 1
3. 계층적 표현 생성 $\rightarrow$ 네트워크가 깊어짐에 따라 패치 병합 레이어를 통해 토큰 수 감소
 - 첫 번째 패치 병합 레이어: 2×2 인접 패치 그룹의 특징 연결, $4C$ 차원의 연결된 특징에 선형 레이어 적용
 $\rightarrow$ 토큰 수 $2 \times 2 = 4$ 배(해상도 2배 다운샘플링)으로 축소, 출력 차원 $= 2C$
 - 이후 Swin Transformer 블록이 특징 변환 위해 적용($H \ 8 \times W \ 8$)
 $\Rightarrow$ 이 첫 번째 패치 병합 및 특징 변환 블록이 Stage 2
 - Stage 3, 4로 두 번 반복(각 $H \ 16 \times W \ 16$, $H \ 32 \times W \ 32$)

- VGG, ResNet 과 같은 일반적인 컨볼루션 네트워크와 동일한 feature map 해상도를 갖는 계층적 표현을 공동으로 생성 → 다양한 비전 작업에서 백본 네트워크 대체 가능

- Swin Transformer block

- 트랜스포머의 MSA(multi-head self attention) 모듈을 shifted window 기반 모듈로 교체해 구축
- shifted window 기반 MSA 모듈, GELU 활성화 함수를 갖는 2 레이어 MLP로 구성됨
- 각 MSA와 MLP 앞에 LayerNorm 레이어, 뒤에 residual connection 적용

- **Shifted Window 기반 self-attention**

- Self-attention in non-overlapped windows

- global self-attention은 이차 복잡도를 가지기 때문에 엄청난 수의 토큰이나 고해상도 이미지 표현해야 하는 많은 비전 문제에 적합 X
- 지역 윈도우 내에서 셀프 어텐션을 계산할 것을 제안

- 각 윈도우가 $M \times M$ 패치 포함한다 가정할 때, $h \times w$ 패치의 이미지에 대한 전역 MSA 모듈, 윈도우 기반 모듈의 계산 복잡도

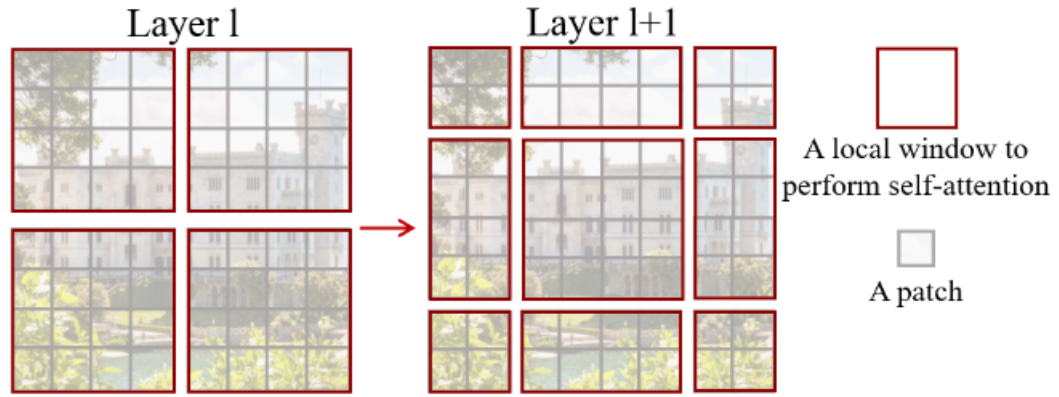
$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C,$$

$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$

→ 전역 셀프 어텐션 계산은 큰 hw 에 대해 부담스러움, 윈도우 기반 셀프 어텐션은 확장 가능

- Shifted window partitioning in successive blocks

- 윈도우 기반 셀프 어텐션 모듈은 윈도우 간의 연결이 부족해 모델링 능력 제한됨
→ 연속적인 Swin Transformer 블록에서 두 가지 partitioning configurations를 번갈아 사용하는 이동된 윈도우 분할 접근 방식 제안



- 첫 번째 모듈은 좌상단 픽셀에서 시작하는 일반적인 윈도우 분할 전략 사용
8x8 feature map은 4x4 크기의 2x2 윈도우로 분할(M=4)
- 다음 모듈은 이전 레이어의 윈도우 구성에서

$$(\lfloor \frac{M}{2} \rfloor, \lfloor \frac{M}{2} \rfloor)$$

픽셀만큼 윈도우를 이동시켜 이동된 윈도우 구성을 채택

- 이동된 윈도우 분할 접근 방식 사용한 연속적인 Swin Transformer 블록

$$\hat{\mathbf{z}}^l = \text{W-MSA}(\text{LN}(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1},$$

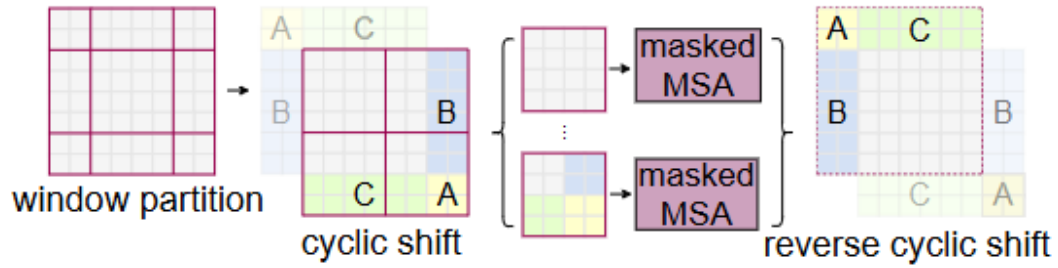
$$\mathbf{z}^l = \text{MLP}(\text{LN}(\hat{\mathbf{z}}^l)) + \hat{\mathbf{z}}^l,$$

$$\hat{\mathbf{z}}^{l+1} = \text{SW-MSA}(\text{LN}(\mathbf{z}^l)) + \mathbf{z}^l,$$

$$\mathbf{z}^{l+1} = \text{MLP}(\text{LN}(\hat{\mathbf{z}}^{l+1})) + \hat{\mathbf{z}}^{l+1},$$

- $\hat{\mathbf{z}}^l, \mathbf{z}^l$ 은 각각 블록 l에 대한 (S)WMSA 모듈과 MLP 모듈의 출력적 특징 나타냄
- W-MSA와 SW-MSA는 각각 일반 및 이동된 윈도우 분할 구성을 사용하는 윈도우 기반 멀티 헤드 셀프 어텐션
- 이동된 윈도우 분할 접근 방식은 이전 레이어의 이웃하는 non-overlapping 윈도우 간의 연결을 도입 → 이미지 분류, 객체 탐지 및 semantic segmentation에서 효과적

- Efficient batch computation for shifted configuration



- 이동된 윈도우 분할의 문제
 - 이동된 구성에서 더 많은 윈도우로 결과 이어지고, 일부 윈도우는 $M \times M$ 보다 작아짐
- 작은 윈도우를 $M \times M$ 크기로 패딩하고 어텐션을 계산할 때 패딩된 값을 마스킹해 해결
- 왼쪽 상단 방향으로 순환 이동하는 효율적인 배치 계산 접근 방식 제안
 - 이동 후 배치된 윈도우는 특징 맵에서 인접하지 않은 여러 하위 윈도우로 구성될 수 있으므로, 마스킹 매커니즘 사용해 셀프 어텐션 계산을 각 하위 윈도우 내로 제한
- 순환 이동을 사용하면 배치된 윈도우의 수가 일반적 윈도우 분할과 동일하게 유지됨

• Relative position bias

- 셀프 어텐션 계산 시 유사도 계산을 위해 각 헤드에 상대 위치 편향 $B \in R^{M^2 \times M^2}$ 을 포함

$$\text{Attention}(Q, K, V) = \text{SoftMax} \left(\frac{QK^T}{\sqrt{d}} + B \right) V,$$

- $Q, K, V \in R^{M^2 \times d}$ 는 쿼리, 키, 값 행렬, d 는 쿼리/키 차원, M^2 은 윈도우 내 패치 수
- 윈도우 내에서 패치의 상대적 위치는 각 축을 따라 $[-M+1, M-1]$ 범위에 있음
- 이러한 상대적 위치 정보를 모두 직접 학습하는 대신 더 작은 크기의 편향 행렬 $\hat{B} \in R^{(2M-1) \times (2M-1)}$ 을 학습
 - 실제 어텐션 계산 시 사용되는 B 행렬의 값을 $\hat{B}$ 에서 가져오므로 파라미터 수 줄여 효율성 높임
- 위치 임베딩 추가 시 성능의 저하가 있어 위치 임베딩은 채택 X

- 사전 학습된 상대 위치 편향은 bi-cubic interpolation을 통해 다른 윈도우 크기에 대한 모델을 미세 조정하기 위해 초기화하는데 사용 가능

- Architecture Variants

- Swin-B: ViTB/DeiT-B와 유사한 모델 크기 및 계산 복잡성 가지는 기본 모델
- Swin-T, Swin-S, Swin-L: 모델 크기 및 계산 복잡성 각각 약 0.25배, 0.5배, 2배
 - Swin-T는 ResNet-50, Swin-S는 ResNet-101과 유사
- 기본 윈도우 크기: $M=7$, 헤드의 쿼리 차원: $d=32$, MLP의 확장 계층: $a=4$

- Swin-T: $C = 96$, layer numbers = $\{2, 2, 6, 2\}$
- Swin-S: $C = 96$, layer numbers = $\{2, 2, 18, 2\}$
- Swin-B: $C = 128$, layer numbers = $\{2, 2, 18, 2\}$
- Swin-L: $C = 192$, layer numbers = $\{2, 2, 18, 2\}$

- C : 첫 번째 단계의 히든 레이어 채널 수

Contribution

1. 새로운 비전 백본 Swin Transformer 제안

- Swin Transformer는 언어 분야의 트랜스포머나 기존의 CNN처럼 컴퓨터 비전 전반에 걸쳐 범용적으로 사용될 수 있는 새로운 비전 트랜스포머를 제안

2. 계층적(Hierarchical) 트랜스포머 구조

- 시각 정보의 다양한 스케일을 모델링하기 위해, 입력 이미지를 패치로 분할한 후 깊은 레이어로 갈수록 인접한 패치를 병합하여 계층적인 특징 맵을 생성하는 새로운 구조를 도입
- 기존 비전 트랜스포머가 단일 해상도의 특징 맵을 생성했던 것과 대조적 구조

3. Shifted Window 기반 셀프 어텐션

- 기존 트랜스포머의 계산 복잡도 문제를 해결하기 위해, 이미지 전체가 아닌 각 국소적인 윈도우 내에서만 셀프 어텐션을 계산하고, 인접한 윈도우 간의 연결을 강화하기 위해 윈도우의 위치를 다음 레이어에서 이동시키는(shifted) 방식을 제안
- 계산 효율성과 모델링 능력의 향상

4. 이미지 크기에 대한 선형 복잡도

- 계층적 구조와 Shifted Window 셀프 어텐션 덕분에 Swin Transformer는 입력 이미지 크기에 대해 선형적인 계산 복잡도 가짐
- 고해상도 이미지나 밀집 예측(dense prediction) 태스크에서 기존 트랜스포머의 이차 복잡도 문제를 해결

5. 다양한 비전 태스크에서의 최신 성능

- Swin Transformer는 이미지 분류, 객체 탐지, 의미론적 분할 등 다양한 비전 태스크에서 기존의 최신 모델들을 크게 능가하는 성능을 달성

4. 실험 및 결과

Dataset

- 이미지 분류: ImageNet-1K, 더 큰 규모의 ImageNet-22K 데이터셋
- 객체 탐지 및 인스턴스 분할: COCO 2017 데이터셋
- 의미론적 분할: ADE20K 데이터셋

Baseline

- 이미지 분류:
 - 기존 Vision Transformer 기반 모델: ViT, DeiT
 - CNN 기반 모델: ResNe(X)t, RegNet, EfficientNet
 - ImageNet-22K 사전 학습 모델: ViT, ResNeXt 등
- 객체 탐지:
 - Cascade Mask R-CNN, ATSS, RepPoints v2, Sparse RCNN 등 다양한 프레임워크에서 CNN (ResNet-50) 및 Swin Transformer 기반 백본 성능 비교
 - Swin Transformer를 백본으로 사용했을 때의 성능을 ResNe(X)t, DeiT-S와 비교
 - 최신 기법들과의 비교를 위해 HTC++ 프레임워크와 함께 Copy-paste, DetectoRS 등과 비교
- 의미론적 분할:
 - UperNet 프레임워크에서 ResNet-101, ResNeSt-101, DeiT-S와 Swin Transformer (Swin-S, Swin-B, Swin-L)의 성능 비교
 - 최신 기법인 SETR과 비교

결과

- Swin Transformer는 세 가지 주요 태스크에서 모두 뛰어난 성능을 보였으며, 기존 최신 모델들을 능가

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [48]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [48]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [48]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [58]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [58]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [58]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [58]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [58]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [63]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [63]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [63]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.5
Swin-B	384 ²	88M	47.0G	84.7	84.5

method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [38]	384 ²	388M	204.6G	-	84.4
R-152x4 [38]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.4
Swin-L	384 ²	197M	103.9G	42.1	87.3

Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [68] and a V100 GPU, following [63].

Method	Backbone	AP ^{box} ₅₀	AP ^{box} ₅₀	AP ^{box} ₇₅	#param.	FLOPs	FPS
Cascade	R-50	46.3	64.3	50.5	82M	739G	18.0
Mask R-CNN	Swin-T	50.5	69.3	54.9	86M	745G	15.3
ATSS	R-50	43.5	61.9	47.0	32M	205G	28.3
	Swin-T	47.2	66.5	51.3	36M	215G	22.3
RepPointsV2	R-50	46.5	64.6	50.3	42M	274G	13.6
	Swin-T	50.0	68.5	54.2	45M	283G	12.0
Sparse	R-50	44.5	63.4	48.2	106M	166G	21.0
R-CNN	Swin-T	47.9	67.3	52.3	110M	172G	18.4

	AP ^{box} ₅₀	AP ^{box} ₅₀	AP ^{box} ₇₅	AP ^{mask} ₅₀	AP ^{mask} ₇₅	#param.	FLOPs	FPS
DeiT-S [†]	48.0	67.2	51.7	41.4	64.2	44.3	80M	889G 10.4
R50	46.3	64.3	50.5	40.1	61.7	43.4	82M	739G 18.0
Swin-T	50.5	69.3	54.9	43.7	66.6	47.1	86M	745G 15.3
X101-32	48.1	66.5	52.4	41.6	63.9	45.2	101M	819G 12.8
Swin-S	51.8	70.4	56.3	44.7	67.9	48.5	107M	838G 12.0
X101-64	48.3	66.4	52.3	41.7	64.0	45.1	140M	972G 10.4
Swin-B	51.9	70.9	56.5	45.0	68.4	48.7	145M	982G 11.6

Method	mini-val AP ^{box} ₅₀	mini-val AP ^{mask} ₅₀	test-dev AP ^{box} ₅₀	test-dev AP ^{mask} ₅₀	#param.	FLOPs
RepPointsV2* [12]	-	-	52.1	-	-	-
GCNet* [7]	51.8	44.7	52.3	45.4	-	1041G
RelationNet++* [13]	-	-	52.7	-	-	-
SpineNet-190 [21]	52.6	-	52.8	-	164M	1885G
ResNeSt-200* [78]	52.5	-	53.3	47.1	-	-
EfficientDet-D7 [59]	54.4	-	55.1	-	77M	410G
DetectoRS* [46]	-	-	55.7	48.5	-	-
YOLOv4 P7* [4]	-	-	55.8	-	-	-
Copy-paste [26]	55.9	47.2	56.0	47.4	185M	1440G
X101-64 (HTC++)	52.3	46.0	-	-	155M	1033G
Swin-B (HTC++)	56.4	49.1	-	-	160M	1043G
Swin-L (HTC++)	57.1	49.5	57.7	50.2	284M	1470G
Swin-L (HTC++)*	58.0	50.4	58.7	51.1	284M	-

Table 2. Results on COCO object detection and instance segmentation. [†]denotes that additional deconvolution layers are used to produce hierarchical feature maps. * indicates multi-scale testing.

• 이미지 분류 (ImageNet-1K):

- Swin Transformer는 DeiT 모델보다 높은 정확도를 달성
 - Swin-T는 DeiT-S보다 +1.5% 높은 81.3%의 top-1 정확도를 기록했습니다
- ImageNet-22K 사전 학습 후 ImageNet-1K에서 미세 조정했을 때, Swin-B는 86.4%의 top-1 정확도를 달성
 - 유사한 추론 처리 속도를 가지는 ViT-B보다 +2.4% 더 높은 성능, Swin-L은 87.3%의 top-1 정확도를 기록

• 객체 탐지 (COCO):

- 다양한 객체 탐지 프레임워크에서 Swin-T는 ResNet-50보다 일관되게 +3.4~4.2의 AP(Average Precision) 향상을 보임
- Swin Transformer는 ResNeXt101-64×4d와 유사한 모델 크기 및 FLOPs를 가지면서도 +3.6 box AP 및 +3.3 mask AP의 성능 향상
- 최신 기법과의 비교했을 때, Swin-L 모델은 COCO test-dev에서 58.7 box AP와 51.1 mask AP를 달성하여 기존 최고 기록보다 +2.7 box AP 및 +2.6 mask AP 높은 성능

ADE20K		val	test	#param.	FLOPs	FPS
Method	Backbone	mIoU	score			
DANet [23]	ResNet-101	45.2	-	69M	1119G	15.2
DLab.v3+ [11]	ResNet-101	44.1	-	63M	1021G	16.0
ACNet [24]	ResNet-101	45.9	38.5	-	-	-
DNL [71]	ResNet-101	46.0	56.2	69M	1249G	14.8
OCRNet [73]	ResNet-101	45.3	56.0	56M	923G	19.3
UperNet [69]	ResNet-101	44.9	-	86M	1029G	20.1
OCRNet [73]	HRNet-w48	45.7	-	71M	664G	12.5
DLab.v3+ [11]	ResNeSt-101	46.9	55.1	66M	1051G	11.9
DLab.v3+ [11]	ResNeSt-200	48.4	-	88M	1381G	8.1
SETR [81]	T-Large [‡]	50.3	61.7	308M	-	-
UperNet	DeiT-S [†]	44.0	-	52M	1099G	16.2
UperNet	Swin-T	46.1	-	60M	945G	18.5
UperNet	Swin-S	49.3	-	81M	1038G	15.2
UperNet	Swin-B [‡]	51.6	-	121M	1841G	8.7
UperNet	Swin-L [‡]	53.5	62.8	234M	3230G	6.2

Table 3. Results of semantic segmentation on the ADE20K val and test set. [†] indicates additional deconvolution layers are used to produce hierarchical feature maps. [‡] indicates that the model is pre-trained on ImageNet-22K.

• 의미론적 분할 (ADE20K):

- Swin-S는 DeiT-S보다 +4.4 mIoU, ResNet-101보다 +4.4 mIoU, ResNeSt-101보다 +2.4 mIoU 더 높은 성능 보임
- ImageNet-22K로 사전 학습된 Swin-L 모델은 53.5 mIoU를 달성, 기존 최고 성능 모델(SETR)보다 +3.2 mIoU 뛰어난 결과

• Ablation study

- Swin Transformer의 핵심 설계 요소인 shifted window 방식과 relative position bias의 효과를 검증

	ImageNet		COCO		ADE20k
	top-1	top-5	AP ^{box}	AP ^{mask}	mIoU
w/o shifting	80.2	95.1	47.7	41.5	43.3
shifted windows	81.3	95.6	50.5	43.7	46.1
no pos.	80.1	94.9	49.2	42.6	43.8
abs. pos.	80.5	95.2	49.0	42.4	43.2
abs.+rel. pos.	81.3	95.6	50.2	43.4	44.0
rel. pos. w/o app.	79.3	94.7	48.2	41.9	44.1
rel. pos.	81.3	95.6	50.5	43.7	46.1

Table 4. Ablation study on the *shifted windows* approach and different position embedding methods on three benchmarks, using the Swin-T architecture. w/o shifting: all self-attention modules adopt regular window partitioning, without *shifting*; abs. pos.: absolute position embedding term of ViT; rel. pos.: the default settings with an additional relative position bias term (see Eq. (4)); app.: the first scaled dot-product term in Eq. (4).

- **Shifted windows:** Shifted window 접근 방식을 사용한 Swin-T는 단일 윈도우만 사용하는 경우보다 ImageNet-1K에서 +1.1% top-1 정확도, COCO에서 +2.8 box AP/+2.2 mask AP, ADE20K에서 +2.8 mIoU 더 높은 성능
- **Relative position bias:** Relative position bias를 사용했을 때, 절대 위치 임베딩이나 위치 인코딩이 없는 경우보다 성능이 향상
특히, 절대 위치 임베딩은 분류 성능에는 도움을 주었으나 객체 탐지 및 의미론적 분할 성능은 저하시킴

method	MSA in a stage (ms)				Arch. (FPS)		
	S1	S2	S3	S4	T	S	B
sliding window (naive)	122.5	38.3	12.1	7.6	183	109	77
sliding window (kernel)	7.6	4.7	2.7	1.8	488	283	187
Performer [14]	4.8	2.8	1.8	1.5	638	370	241
window (w/o shifting)	2.8	1.7	1.2	0.9	770	444	280
shifted window (padding)	3.3	2.3	1.9	2.2	670	371	236
shifted window (cyclic)	3.0	1.9	1.3	1.0	755	437	278

Table 5. Real speed of different self-attention computation methods and implementations on a V100 GPU.

	Backbone	ImageNet		COCO		ADE20k
		top-1	top-5	AP ^{box}	AP ^{mask}	mIoU
sliding window	Swin-T	81.4	95.6	50.2	43.5	45.8
Performer [14]	Swin-T	79.0	94.2	-	-	-
shifted window	Swin-T	81.3	95.6	50.5	43.7	46.1

Table 6. Accuracy of Swin Transformer using different methods for self-attention computation on three benchmarks.

- **Shifted window와 sliding window 비교:** 시프트 윈도우(shifted window) 방식은 슬라이딩 윈도우(sliding window) 방식보다 계산 속도가 월등히 빠르며(최대

40.8배), Performer와 같은 다른 빠른 트랜스포머 방식과 비교해도 속도와 정확도 면에서 우수한 성능을 보임

⇒ Swin Transformer가 다양한 컴퓨터 비전 태스크에서 효율적이고 강력한 일반 목적 백본으로 활용될 수 있음을 입증

5. 결론 (배운점)

- 의의
 - 효율적인 계층적 특징 학습
 - 패치 병합(patch merging) 레이어를 통해 자연스럽게 계층적 특징을 표현
 - Shifted Window 기반 Self-Attention
 - Swin Transformer는 국소 창(local window) 내 어텐션을 계산하고 이를 Shifted Window 방식으로 연결 → 계산 복잡도가 이미지 크기에 대해 선형($O(N)$)
 - 다양한 태스크에서의 SOTA 달성
 - 이미지 분류, 객체 탐지, 의미론적 분할 등 주요 비전 벤치마크에서 기존 최신 모델 대비 상당한 성능 향상을 달성하며, 트랜스포머 기반 모델이 비전 분야의 표준 백본이 될 수 있음 보여줌
- 한계점
 - 실제 하드웨어(특히 GPU)에서의 메모리 접근 및 연산 최적화
 - 실제 구현 및 커널 최적화가 성능에 영향을 미칠 수 있음을 언급
 - 대규모 사전 학습 데이터의 필요성
 - 높은 성능을 달성하기 위해 ImageNet-1K와 같은 데이터셋 외에 ImageNet-22K와 같은 더 큰 규모의 데이터셋으로 사전 학습됨
 - 위치 정보 표현 방식의 효과
 - 상대 위치 편향(relative position bias)이 효과적임을 보였으나, 절대 위치 임베딩(absolute position embedding)이 오히려 일부 태스크(객체 탐지, 의미론적 분할)에서 성능을 저하시키는 경향 → 트랜스포머 모델에서 위치 정보 표현 방식이 태스크에 따라 민감할 수 있음



- Swin Transformer는 CNN의 계층적 특징 학습 능력과 트랜스포머의 장기 의존성 모델링 능력을 효과적으로 결합한 연구라고 생각한다.
- shifted window라는 아이디어를 통해 윈도우 간의 연결성을 크게 높여 모델의 표현력을 강화한다는 점이 인상깊었다.
- 상대 위치 편향이 효과적임을 검증한 것을 넘어 어떤 방식으로 위치 정보를 모델링하는 것이 다양한 비전 태스크에 가장 적합한지, 절대 위치 임베딩이 오히려 성능을 저하시킨다는 결과의 이유가 무엇인지 등 위치 정보 표현에 있어서 추가적인 분석이 더해진다면 더 좋을 것 같다.